Underactuated Robotics

Algorithms for Walking, Running, Swimming, Flying, and Manipulation

Russ Tedrake

Last modified 2024-11-11.
How to cite these notes, use annotations, and give feedback.

Note: These are working notes used for a course being taught at MIT. They will be updated throughout the Spring 2024 semester. Lecture videos are available on YouTube.

Previous Chapter | Table of contents | Next Chapter

Launch in Deepnote

Appendix C Optimization and Mathematical Programming

The view of dynamics and controls taken in these notes builds heavily on tools from optimization -- and our success in practice depends heavily on the effective application of numerical optimization. There are many excellent books on optimization, for example [1] is an excellent reference on smooth optimization and [2] is an excellent reference on convex optimization (which we use extensively). I will provide more references for the specific optimization formulations below.

The intention of this chapter is, therefore, mainly to provide a launchpad and to address some topics that are particularly relevant in robotics but might be more hidden in the existing general treatments. You can get far very quickly as a consumer of these tools with just a high-level understanding. But, as with many things, sometimes the details matter and it becomes important to understand what is going on under the hood. We can often formulate an optimization problem in multiple ways that might be mathematically equivalent, but perform very differently in practice.

C.1 Optimization software

Some of the algorithms from optimization are quite simple to implement yourself; stochastic gradient descent is perhaps the classic example. Even more of them are conceptually fairly simple, but for some of the algorithms, the implementation details matter a great deal -- the difference between an expert implementation and a novice implementation of the numerical recipe, in terms of speed and/or robustness, can be dramatic. These packages often use a wealth of techniques for numerically conditioning the problems, for discarding trivially valid constraints, and for warm-starting optimization between solves. Not all solvers support all types of objectives and constraints, and even if we have two commercial solvers that both claim to solve, e.g. quadratic programs, then they might perform differently on the particular structure/conditioning of your problem. In some cases, we also have nicely designed open-source alternatives to these commercial solvers (often written by academics who are experts in optimization) -- sometimes they compete well with the commercial solvers or even outperform them in particular problem types.

As a result, there are also a handful of software packages that attempt to provide a layer of abstraction between the formulation of a mathematical program and the instantiation of that problem in each of the particular solvers. Famous examples of this include CVX and YALMIP for MATLAB, and JuMP for Julia. DRAKE's MathematicalProgram classes provide a middle layer like this for C++ and Python; its creation was motivated initially and specifically by the need to support the optimization formulations we use in this text.

We have a number of tutorials on mathematical programming in DRAKE, starting with a general introduction here. Drake supports a number of custom, open-source, and commercial solvers (and even some of the commercial solvers are free to academic users).

C.2 GENERAL CONCEPTS

The general formulation is ...

It's important to realize that even though this formulation is incredibly general, it does have it's limits. As just one example, when write optimizations to plan trajectories of a robot, in this formulation we typically have to choose a-priori as particular number of decision variables that will encode the solution. Although we can of course write algorithms that change the number of variables and call the optimizer again, somehow I feel that this "general" formulation fails to capture, for instance, the type of mathematical programming that occurs in sample-based optimization planning -- where the resulting solutions can be described by any finite number of parameters, and the computation flexibly transitions amongst them.

C.2.1 Convex vs nonconvex optimization

Local optima. Convex functions, convex constraints.

If an optimization problem is nonconvex, it does not necessarily mean that the optimization is hard. There are many cases in deep learning where we can reliably solve seemingly very high-dimensional and nonconvex optimization problems. Our understanding of these phenomenon is evolving rapidly, and I suspect anything I write here will shortly become outdated. But the current thinking in supervised learning mostly revolves around the idea that over-parameterization is key -- that many of these success stories are happening in a regime where we actually have more decision variables than we have data, and that the search space is actually dense with solutions that can fit the data perfectly (the so-called "interpolating solutions"), that not all global minima are equally robust, and that the optimization algorithms are performing some form of implicit regularization in order to pick a "good" interpolating solution.

C.2.2 Constrained optimization with Lagrange multipliers

Given the equality-constrained optimization problem

$$\text{minimize}_{\mathbf{z}} \ell(\mathbf{z}) \quad \text{subject to} \quad \phi(\mathbf{z}) = 0,$$

where ϕ is a vector. Define a vector λ of Lagrange multipliers, the same size as ϕ, and the scalar Lagrangian function,

$$L(\mathbf{z}, \lambda) = \ell(\mathbf{z}) + \lambda^T \phi(\mathbf{z}).$$

A necessary condition for $\mathbf{z}^*$ to be an optimal value of the constrained optimization is that the gradients of L vanish with respect to both $\mathbf{z}$ and λ:

$$\frac{\partial L}{\partial \mathbf{z}} = 0, \quad \frac{\partial L}{\partial \lambda} = 0.$$

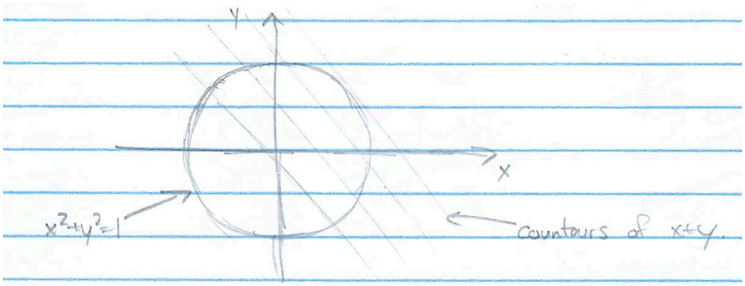
Note that $\frac{\partial L}{\partial \lambda} = \phi(\mathbf{z})$, so requiring this to be zero is equivalent to requiring the constraints to be satisfied.

Example C.1 (Optimization on the unit circle)

Consider the following optimization:

$$\min_{x,y} x + y, \quad \text{subject to} \quad x^2 + y^2 = 1.$$

The level sets of $x + y$ are straight lines with slope -1, and the constraint requires that the solution lives on the unit circle.



Simply by inspection, we can determine that the optimal solution should be $x = y = -\frac{\sqrt{2}}{2}$. Let's make sure we can obtain the same result using Lagrange multipliers.

Formulating

$$L = x + y + \lambda(x^2 + y^2 - 1),$$

we can take the derivatives and solve

$$\begin{aligned} \frac{\partial L}{\partial x} &= 1 + 2\lambda x = 0 \quad &\Rightarrow \quad \lambda &= -\frac{1}{2x}, \\ \frac{\partial L}{\partial y} &= 1 + 2\lambda y = 1 - \frac{y}{x} = 0 \quad &\Rightarrow \quad y &= x, \\ \frac{\partial L}{\partial \lambda} &= x^2 + y^2 - 1 = 2x^2 - 1 = 0 \quad &\Rightarrow \quad x &= \pm\frac{1}{\sqrt{2}}. \end{aligned}$$

Given the two solutions which satisfy the necessary conditions, the negative solution is clearly the minimizer of the objective.

C.3 Convex optimization

C.3.1 Linear Programs/Quadratic Programs/Second-Order Cones

Example: Balancing force control on Atlas

C.3.2 Semidefinite Programming and Linear Matrix Inequalities

Semidefinite programming relaxation of general quadratic optimization

One of the most powerful use cases for semidefinite programming is in writing convex relaxations of more general non-convex optimization problems. One particularly useful example of this is the general quadratic optimization problem, written as

$$\begin{aligned} \min_{\mathbf{y}} \quad & \mathbf{x}^T \mathbf{Q} \mathbf{x} \\ \text{subject to} \quad & \mathbf{x}^T \mathbf{A}_i \mathbf{x} \geq 0, \\ & \mathbf{B}\mathbf{x} \geq \mathbf{0}, \\ & \mathbf{x} = \begin{bmatrix} 1 \\ \mathbf{y} \end{bmatrix}. \end{aligned}$$

In the following, we won't make any assumptions about $\mathbf{Q}$ or $\mathbf{A}_i$ being PSD. For instance, quadratic equality constraints of the form $\mathbf{x}^T \mathbf{A} \mathbf{x} = 0$ (which are generically nonconvex) can be encoded with two constraints: $\mathbf{x}^T \mathbf{A} \mathbf{x} \geq 0$ and $-\mathbf{x}^T \mathbf{A} \mathbf{x} \geq 0$. The SDP relaxation [3, 4] is given by

$$\begin{aligned} \min_{\mathbf{X}} \quad & \operatorname{tr}(\mathbf{Q}\mathbf{X}) \\ \text{subject to} \quad & \mathbf{e}_1^T \mathbf{X} \mathbf{e}_1 = 1 \\ & \operatorname{tr}(\mathbf{A}_i \mathbf{X}) \geq 0, \\ & \mathbf{B}\mathbf{X}\mathbf{e}_1 \geq \mathbf{0}, \\ & \mathbf{B}\mathbf{X}\mathbf{B}^T \geq \mathbf{0}, \\ & \mathbf{X} \succeq 0, \end{aligned}$$

where $\mathbf{e}_1$ is the vector with 1 on the first element and the other elements all zero. Let's try to understand the power of this relaxation through a few simple examples.

Example C.2 (Semidefinite programming relaxation of non-convex quadratic constraints)

Consider the problem:

$$\begin{aligned} \min_{x} \quad & \|x - a\|^2 \\ \text{subject to} \quad & \|x - b\|^2 \geq 1 \end{aligned}$$

This problem is non-convex, however we can write a convex relaxation of the problem using the semidefinite-programming relaxation:

$$\begin{aligned} \min_{x,y} \quad & y - 2ax + a^2 \\ \text{subject to} \quad & y - 2bx + b^2 \geq 1 \\ & y \geq x^2 \end{aligned}$$

where we write $y \geq x^2$ as the semidefinite constraint $\begin{bmatrix} y & x \\ x & 1 \end{bmatrix} \succeq 0$. †

† Note th particular constrain simple th have alte been writ second-o constrain

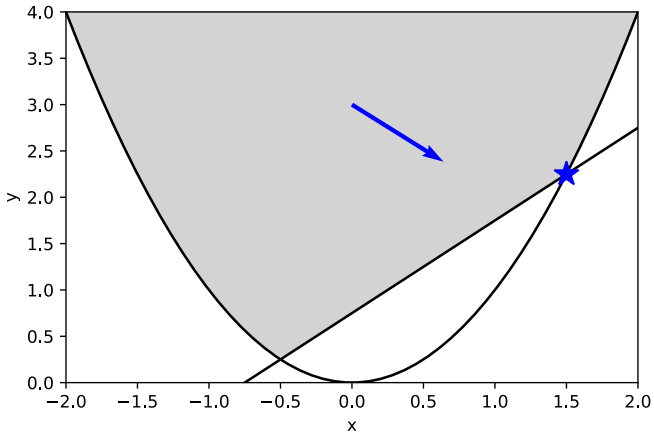


Figure C.2 - Optimization landscape for $a = .8, b = .5$.

I've plotted the feasible region and the objective with an arrow. As you can see, the feasible region is the epigraph of $y = x^2$ intersected with the linear constraint. Now here is the key point: for linear objectives, the optimal solution will (almost) never be active on the linear constraint unless it's at a vertex of the feasible set (this is related to the classical picture from linear programming). In this example specifically, the constraint $y = x^2$ will be inactive only if $a = b$; for every other objective, this relaxation will give $y = x^2$, and will give the optimal solution to the original problem. Note that we could have equally well written a quadratic equality constraint.

Launch in Deepnote

Example C.3 (Semidefinite programming relaxation of the unit circle)

Another useful example is

$$\begin{aligned} \min_{\mathbf{x}} \quad & \mathbf{x}^T \mathbf{A} \mathbf{x} + \mathbf{b}^T \mathbf{x}, \\ \text{subject to} \quad & \mathbf{x}^T \mathbf{x} = 1. \end{aligned}$$

Again this optimization is non-convex, but we can write the semidefinite-programming relaxation as

$$\begin{aligned} \min_{\mathbf{Y}, \mathbf{x}} \quad & \operatorname{tr}(\mathbf{A}\mathbf{Y}) + \mathbf{b}^T \mathbf{x}, \\ \text{subject to} \quad & \operatorname{tr}(\mathbf{Y}) = 1, \\ & \mathbf{Y} \succeq \mathbf{x}\mathbf{x}^T, \end{aligned}$$

where the PSD constraint is again written using the Schur complement. For the case where $\mathbf{x} \in \mathbb{R}^2$, the PSD constraint gives us that $Y_{11} \geq x_1^2$ and $Y_{22} \geq x_2^2$; combined with the trace constraint this gives us

$$Y_{22} = 1 - Y_{11} \geq x_2^2 \Rightarrow 1 - x_2^2 \geq Y_{11} \geq x_1^2 \Rightarrow 1 \geq x_1^2 + x_2^2.$$

Once again the picture looks like the affine section of a cone (now in higher dimensions). Given the linear objective, we expect the optimal solution to be unbounded, or active on one ore more of the constraints. Whenever the optimal solution is unique, the optimization will obtain the rank one solution for the PSD constraint (the matrix Y is rank one, and the Schur complement matrix is also rank

one), which implies that $\mathbf{Y} = \mathbf{x}\mathbf{x}^T$ and we have $\mathbf{x}^T\mathbf{x} = 1$. When the convex relaxation gives the optimal solution to the original problem, we say that the convex relaxation is "tight".

Let's compare this to the simpler relaxation,

$$\begin{aligned} \min_{\mathbf{x}} \quad & \mathbf{x}^T\mathbf{A}\mathbf{x} + \mathbf{b}^T\mathbf{x}, \\ \text{subject to} \quad & \mathbf{x}^T\mathbf{x} \leq 1, \end{aligned}$$

which can be written as a second-order cone program (SOCP). When the objective is linear $(\mathbf{A} = 0, \mathbf{b} \neq 0)$, even this simpler relaxation will be tight. But when the objective is quadratic, if the minimum of the quadratic falls inside the unit disk, then the relaxation will not be tight (it will be "loose") -- the optimal solution will satisfy the constraints $\mathbf{x}^T\mathbf{x} \leq 1$, but will not have $\mathbf{x}^T\mathbf{x} = 1$. In the PSD relaxation, the convex relaxation will be tight, because the objective is now linear (in the lifted coordinates), and will push towards the boundary so long as $\mathbf{b} \neq 0$.

I've given a numerical example you can play with in the notebook:

Launch in Deepnote

What's amazing is that this SDP relaxation even gives the optimal solution when the objective is non-convex. I've given an example in the notebook with a concave objective (e.g. $\mathbf{A} \preceq 0$).

C.3.3 Sums-of-squares optimization

It turns out that in the same way that we can use SDP to search over the positive quadratic equations, we can generalize this to search over the positive polynomial equations. To be clear, for quadratic equations we have

$$\mathbf{P} \succeq 0 \quad \Rightarrow \quad \mathbf{x}^T\mathbf{P}\mathbf{x} \geq 0, \quad \forall \mathbf{x}.$$

It turns out that we can generalize this to

$$\mathbf{P} \succeq 0 \quad \Rightarrow \quad \mathbf{m}^T(\mathbf{x})\mathbf{P}\mathbf{m}(\mathbf{x}) \geq 0, \quad \forall \mathbf{x},$$

where $\mathbf{m}(\mathbf{x})$ is a vector of polynomial equations, typically chosen as a vector of *monomials* (polynomials with only one term). The set of positive polynomials parameterized in this way is exactly the set of polynomials that can be written as a *sum of squares*[5]. While it is known that not all positive polynomials can be written in this form, much is known about the gap. For our purposes this gap is very small (papers have been written about trying to find polynomials which are uniformly positive but not sums of squares); we should remember that it exists but it won't hinder too many of our use cases.

Even better, there is quite a bit that is known about how to choose the terms in $\mathbf{m}(\mathbf{x})$. For example, if you give me the polynomial

$$p(x) = 2 - 4x + 5x^2$$

and ask if it is positive for all real x, I can convince you that it is by producing the sums-of-squares factorization

$$p(x) = 1 + (1 - 2x)^2 + x^2,$$

or the factorization

$$p(x) = \begin{bmatrix} 1 \\ x \end{bmatrix}^T \begin{bmatrix} 2 & -2 \\ -2 & 5 \end{bmatrix} \begin{bmatrix} 1 \\ x \end{bmatrix},$$

since the matrix in this quadratic form is PSD. In general, I can produce a monomial vector containing all monomials with degree up to the the half of the degree of p (we can actually do much better, but I will not cover those details). In practice, what this means for us is that people have authored optimization front-ends which take a high-level specification with constraints on the positivity of polynomials and they automatically generate the SDP problem for you, without having to think about what terms should appear in $\mathbf{m}(\mathbf{x})$ (c.f. [6]). This class of optimization problems is called Sums-of-Squares (SOS) optimization.

As it happens, the particular choice of $\mathbf{m}(\mathbf{x})$ can have a huge impact on the numerics of the resulting semidefinite program (and on your ability to solve it with commercial solvers). DRAKE implements some particularly novel/advanced algorithms to make this work well [7].

We will write optimizations using sums-of-squares constraints as

$$p(\mathbf{x}) \text{ is SOS}$$

as shorthand for the constraint that $p(\mathbf{x}) \geq 0$ for all $\mathbf{x}$, as demonstrated by finding a sums-of-squares decomposition.

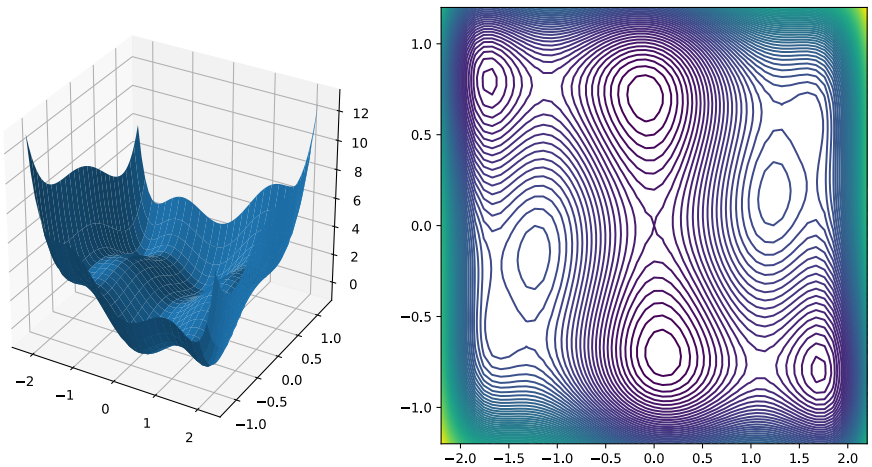
This is surprisingly powerful. It allows us to use convex optimization to solve what appear to be very non-convex optimization problems.

Example C.4 (Global minimization via SOS)

Consider the following famous non-linear function of two variables (called the "six-hump-camel":

$$p(x) = 4x^2 + xy - 4y^2 - 2.1x^4 + 4y^4 + \frac{1}{3}x^6.$$

This function has six local minima, two of them being global minima [8].



By formulating a simple sums-of-squares optimization, we can actually find the minimum value of this function (technically, it is only a lower bound, but in this case and many cases, it is surprisingly tight) by writing:

$$\begin{aligned} \max_{\lambda} \quad & \lambda \\ \text{s.t. } & p(x) - \lambda \text{ is sos.} \end{aligned}$$

Go ahead and play with the code (most of the lines are only for plotting; the actual optimization problem is nice and simple to formulate).

Launch in Deepnote

Note that this finds the minimum value, but does not immediately produce the $\mathbf{x}$ value which minimizes it. To extract the minimizer, one needs to examine the dual program (see, for instance [8]). In many applications, such as Lyapunov analysis, obtaining the minimizer is not even necessary.

Sums of squares on a Semi-Algebraic Set

The S-procedure.

Sums of squares optimization on an Algebraic Variety

The S-procedure

Using the quotient ring

Quotient rings via sampling

DSOS and SDSOS

C.3.4 Solution techniques

Interior point (Gurobi, Mosek, Sedumi, ...), First order methods

C.4 Nonlinear programming

The generic formulation of a nonlinear optimization problem is

$$\min_{z} c(z) \quad \text{subject to} \quad \phi(z) \leq 0,$$

where z is a vector of *decision variables*, c is a scalar *objective function* and ϕ is a vector of *constraints*. Note that, although we write $\phi \leq 0$, this formulation captures positivity constraints on the decision variables (simply multiply the constraint by -1) and equality constraints (simply list both $\phi \leq 0$ and $-\phi \leq 0$) as well.

The picture that you should have in your head is a nonlinear, potentially non-convex objective function defined over (multi-dimensional) z, with a subset of possible z values satisfying the constraints.

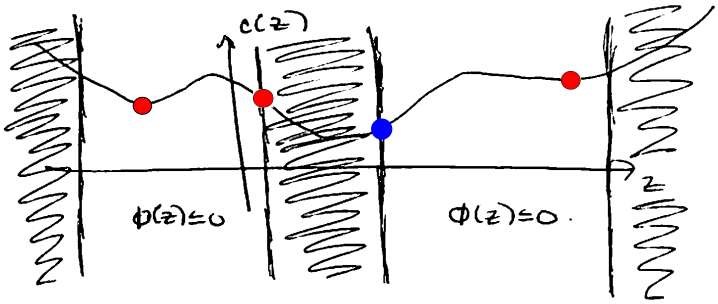


Figure C.3 - One-dimensional cartoon of a nonlinear optimization problem. The red dots represent local minima. The blue dot represents the optimal solution.

Note that minima can be the result of the objective function having zero derivative *or* due to a sloped objective up against a constraint.

Numerical methods for solving these optimization problems require an initial guess, $\hat{z}$, and proceed by trying to move down the objective function to a minima. Common approaches include *gradient descent*, in which the gradient of the objective function is computed or estimated, or second-order methods such as *sequential quadratic programming (SQP)* which attempts to make a local quadratic approximation of the objective function and local linear approximations of the constraints and solves a quadratic program on each iteration to jump directly to the minimum of the local approximation.

While not strictly required, these algorithms can often benefit a great deal from having the gradients of the objective and constraints computed explicitly; the alternative is to obtain them from numerical differentiation. Beyond pure speed considerations, I strongly prefer to compute the gradients explicitly because it can help avoid numerical accuracy issues that can creep in with finite difference methods. The desire to calculate these gradients will be a major theme in the discussion below, and we have gone to great lengths to provide explicit gradients of our provided functions and automatic differentiation of user-provided functions in DRAKE.

When I started out, I was of the opinion that there is nothing difficult about implementing gradient descent or even a second-order method, and I wrote all of the solvers myself. I now realize that I was wrong. The commercial solvers available for nonlinear programming are substantially higher performance than anything I wrote myself, with a number of tricks, subtleties, and parameter choices that can make a huge difference in practice. Some of these solvers can exploit sparsity in the problem (e.g., if the constraints operate in a sparse way on the decision variables). Nowadays, we make heaviest use of SNOPT [9], which now comes bundled with the binary distributions of DRAKE, but also support a large suite of numerical solvers. Note that while I do advocate using these tools, you do not need to use them as a black box. In many cases you can improve the optimization performance by understanding and selecting non-default configuration parameters.

C.4.1 Second-order methods (SQP / Interior-Point)

C.4.2 First-order methods (SGD / ADMM)

Penalty methods

Augmented Lagrangian

Projected Gradient Descent

C.4.3 Zero-order methods (CMA)

C.4.4 Example: Inverse Kinematics

C.5 Mixed-discrete (combinatorial) and continuous optimization

C.5.1 Search, SAT, First order logic, SMT solvers, LP interpretation

C.5.2 Mixed-integer convex optimization

An advanced, but very readable book on MIP [10]. Nice survey paper on MILP [11].

C.5.3 Graphs of Convex Sets

Graphs of Convex Sets (GCS) is a general optimization framework for combining combinatorial optimization problems that are naturally described on a graph (e.g. shortest path problems [12], traveling salesperson, minimum spanning tree, facility location, etc) with continuous optimization. It was originally introduced in [13], and we have a mature implementation in Drake. GCS extensions to the "shortest path problem" are very natural in optimal control, so I will use the shortest path problem as the running example here. See [14] for an excellent and much more complete treatment of GCS as an optimization framework.

Shortest path problems

Consider the very classical problem of finding the (weighted) shortest path from a source, s, to a target, t, on a graph, pictured on the left. The problem is described by a set of vertices, a set of (potentially directed) edges, and the cost of traversing each edge.

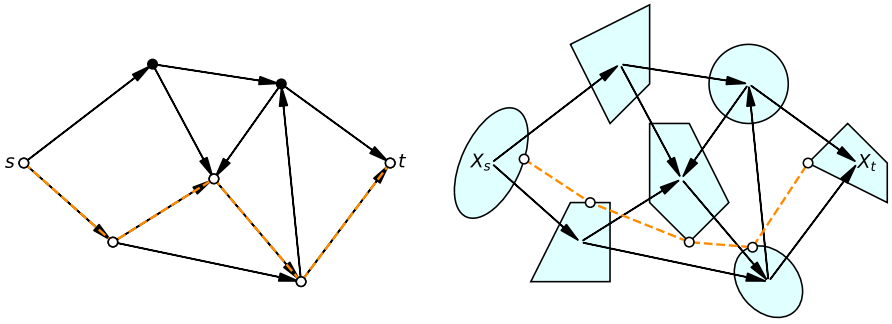


Figure C.4 - Left: A shortest path problem (from source s to target t) on a standard directed graph. Right: The generalization to a graph of convex sets.

It is well known that the discrete graph search can be formulated as a linear program. Define the graph with vertices V and edges $(i,j) \in E$, with costs $c_{ij} \geq 0$ corresponding to the edge (i,j). Define decision variables φ_{ij} where we seek a solution where $\varphi_{ij} = 1$ if the edge (i,j) is in the shortest path, otherwise $\varphi_{ij} = 0$. We can write the linear program as

$$\begin{aligned} \min_{\varphi} \quad & \sum_{(i,j)\in E} c_{ij}\varphi_{ij} & & \text{(path} \\ \text{s.t.} \quad & \sum_{j\in E_i^{out}} \varphi_{ij} + \delta_{ti} = \sum_{j\in E_i^{in}} \varphi_{ji} + \delta_{si}, & \forall i \in V, \quad & \text{(flow co} \\ & \varphi_{ij} \in \{0,1\}, & \forall (i,j) \in E. \quad & \text{(binary c} \end{aligned}$$

The (path length) is simply the sum of the edge costs for all edges in the solution path. The (flow constraints) are a clever way of encoding that the sum of the flows in is equal to the sum of the flows out (except at the source and the target, which are modified by δ_{si} and δ_{ti}, respectively).

GCS provides a simple, but powerful generalization to this problem: whenever we visit a vertex in the graph, we are also allowed to pick one element out of a convex set associated with that vertex. Edge lengths are allowed to be convex functions of the continuous variables in the corresponding sets, and we can also write convex constraints on the edges (which must be satisfied by any solution path). For the generalization of the shortest-path problem above, we now have:

$$\begin{aligned} \min_{\varphi, x} \quad & \sum_{(i,j)\in E} \ell_{ij}(x_i, x_j)\varphi_{ij} \\ \text{s.t.} \quad & x_i \in X_i, && \forall i \in V, \\ & \sum_{j\in E_i^{out}} \varphi_{ij} + \delta_{ti} = \sum_{j\in E_i^{in}} \varphi_{ji} + \delta_{si} \leq 1, && \forall i \in V, \\ & \varphi_{ij} \in \{0,1\}, && \forall (i,j) \in E. \end{aligned}$$

Here $\ell_{ij} \geq 0$ is the edge length, which can be a convex function of the associated vertex variables, and X_i is the bounded convex set associated with vertex i. The `GraphOfConvexSets` implementation in Drake also allows you to add additional convex constraints on the vertices and the edges.

GCS is relatively new, but much is known about the integer programming formulation of the original shortest path problem. In particular, it is well known that if all costs are positive, then if we take the natural convex relaxation of the last line, replacing $\varphi_{ij} \in \{0,1\}$ with $\varphi_{ij} \in [0,1]$, then this convex relaxation \emph{is always tight}. The solution to this linear program will give the optimal solution to the shortest path problem! Technically, in this particular instance, we can even relax the last line to $\varphi_{ij} \geq 0$.

Although the shortest path problem on a graph of convex sets as written has a bilinear objective, it can naturally be described by a mixed-integer convex optimization problem (MICP). Indeed, GCS can encode problems that are NP-Hard, but once again we can study the tightness of the natural convex relaxation of the binary variables. What makes the framework so powerful is that we have developed a convex relaxation for GCS which is both efficient and very strong. This means that you can solve GCS problems to global optimality orders of magnitude faster than previous transcriptions. But in practice, we find that *solving only the convex relaxation* (plus a little rounding) is almost always sufficient to recover the optimal solution. Nowadays, we almost never solve the full MICP in our robotics applications of GCS.

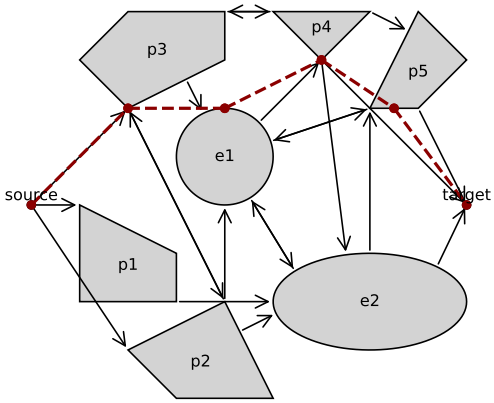
Being able to solve combinatorial problems without an MIP solver is a big deal for me in the context of these notes: the best solvers for MIP are all commercially licensed (and very expensive). Saying that we can solve hard instances with only convex optimization also means that we can solve them with open-sourced solvers.

▶ Expand this if you want a *few* more details about how/why GCS works.

Example C.5 (An example Graph of Convex Sets in 2D)

I've put together a toy GCS problem in two dimensions using a variety of different sets. By solving the convex relaxation, we can obtain a lower boundo on the total

cost (path length). They by rounding we can find a feasible solution, which gives an upper bound on the total cost. Since they are equal up to numerical tolerances, in this problem we can confirm -- without ever solving the mixed-integer version of the problem -- that the convex relaxation was tight.



Launch in Deepnote

Applications

Here are some back-references to applications of GCS throughout these notes:

- Trajectory optimization, e.g.
  - Minimum-time linear optimal control,
  - Piecewise-affine MPC (PWAMPC),
  - Quadrotor planning around obstacles (with differential flatness),
- Planning through contact, and
- Footstep planning for humanoids and quadrupeds.

You can also find more applications of GCS, especially for *kinematic* trajectory optimization, in my manipulation notes.

C.6 "Black-box" optimization

Derivative-free methods. Some allow noisy evaluations.

References

1. Jorge Nocedal and Stephen J. Wright, "Numerical optimization", Springer , 2006.

2. Stephen Boyd and Lieven Vandenberghe, "Convex Optimization", Cambridge University Press , 2004.

3. Pablo Parrilo, "Lecture notes from MIT 6.7230 - Algebraic techniques and semidefinite optimization", , 2023.

4. Jaehyun Park and Stephen Boyd, "General heuristics for nonconvex quadratically constrained quadratic programming", *arXiv preprint arXiv:1703.07870*, 2017.

5. Pablo A. Parrilo, "Structured Semidefinite Programs and Semialgebraic Geometry Methods in Robustness and Optimization", PhD thesis, California Institute of Technology, May 18, 2000.

6. Stephen Prajna and Antonis Papachristodoulou and Peter Seiler and Pablo A. Parrilo, "SOSTOOLS: Sum of Squares Optimization Toolbox for MATLAB User's guide", , June 1, 2004.

7. Frank Noble Permenter, "Reduction methods in semidefinite and conic optimization", PhD thesis, Massachusetts Institute of Technology, 2017. [link]

8. Didier Henrion and Jean-Bernard Lasserre and Johan Lofberg, "GloptiPoly 3: moments, optimization and semidefinite programming", *Optimization Methods & Software*, vol. 24, no. 4-5, pp. 761--779, 2009.

9. Philip E. Gill and Walter Murray and Michael A. Saunders, "User's Guide for SNOPT Version 7: Software for Large -Scale Nonlinear Programming", , February 12, 2006.

10. Michele Conforti and Gérard Cornuéjols and Giacomo Zambelli and others, "Integer programming", Springer , vol. 271, 2014.

11. Juan Pablo Vielma, "Mixed integer linear programming formulation techniques", *Siam Review*, vol. 57, no. 1, pp. 3--57, 2015.

12. Ravindra K Ahuja and Thomas L Magnanti and James B Orlin, "Network flows: theory, algorithms, and applications", Prentice-Hall, Inc. , 1993.

13. Tobia Marcucci and Jack Umenberger and Pablo A. Parrilo and Russ Tedrake, "Shortest Paths in Graphs of Convex Sets", *arxiv*, 2023. [link]

14. Tobia Marcucci, "Graphs of Convex Sets with Applications to Optimal Control and Motion Planning", PhD thesis, Massachusetts Institute of Technology, May, 2024. [link]